Presentation 2

STAT 390 Fall 2026

Team 2 · The Consultancy

Lauren Vegas

Callie Berthold

Lily Wu

Alyssa Kantner

Izabella Kulczycka

Analysts · Choice 1

Before · Scores were matched to images by sort order

Like taking the name tags off photos, then putting them back in a different order.

1 Two lists, two orders

Data prep lists patch files alphabetically, so patch10 comes right after patch1.

attention_analysis.py re-sorted them by number, so patch2 comes after patch1.

The DataLoader dropped the filenames, so the score had no name attached. **No error is raised, and the image still looks like tissue.**

Score #	File the model scored	File shown	Same?
1	patch0	patch0	✓
2	patch1	patch1	✓
3	patch10	patch2	✗
4	patch11	patch3	✗

Illustration of the two orders (filenames shortened).

The attention numbers themselves were correct; only the image shown next to each number could be wrong.

Before vs. after · How a score finds its image

1 Before: matched by guessing the order

dataset.py
tensor only

→

model
score #1, #2, #3…

→

analysis
re-sorts the folder

→

image
can be the wrong patch

2 After: matched by filename

dataset.py
tensor + filenames

→

model
score i ↔ file i

→

analysis
opens that file

→

image
the patch that was scored

Nothing is re-sorted anywhere: the filename travels with its patch from the dataset to the figure.

Before vs. after · Change 1: dataset.py

1 Before

```
try:
  v = torch.load(fp)
  vecs.append(v)
except Exception:
  continue  # silently skipped

return torch.stack(vecs)
# filenames are gone

# __getitem__
return {..., "stain_slices": ...}
```

2 After

```
missing = [p for p in patch_paths
           if no .pt file exists]
if missing:
  print("EXCLUDED", ...) # listed
  drop them from patch_paths

return torch.stack(vecs), patch_paths
# row i = patch_paths[i]

# __getitem__
return {..., "stain_paths": stain_paths}
```

Function: _load_slice_tensor in dataset.py; __getitem__ adds stain_paths to each case.

Before vs. after · Change 2: attention_analysis.py

1 Before

```
png_path = _lookup_png_for_patch(
  case_id, stain, slice_idx, patch_idx)
# re-globs and re-sorts the folder

try:
  ax.imshow(Image.open(png_path))
except Exception:
  _plot_feature_heatmap(ax, vec)
  # fallback, no error
```

2 After

```
stain_paths = case_data["stain_paths"]
check_alignment(...)
# number of scores = number of files

"patch_path": path,
# stored next to its score

ax.imshow(load_patch_image(path))
# missing file → error
```

Removed: _lookup_png_for_patch, the folder-index helpers and the heatmap fallback. New output: patch_attention_by_filename.csv.

Before vs. after · Silent failures now stop the run

If…	Before	After
An embedding (.pt) was never made (patch < 32×32)	patch dropped, no message	patch left out and listed in the log
An embedding (.pt) is unreadable	patch skipped	error naming the file
A slice has no patches	slice skipped	error
A case has no label	case skipped	error
A patch image (.png) is missing	feature heatmap shown instead	error naming the file
Number of scores ≠ number of filenames	not checked	error

Nothing is dropped without a message: a patch is either scored with its filename, listed as excluded, or the run stops.

The 32×32 minimum is set in precompute_pooled_features.py (MIN_PATCH_SIZE = 32), so these patches never had an embedding.

Finding · The new check stopped the whole run

Training job FAILED after 1 min 21 s, on the first patch with no embedding

The old code would have skipped this patch silently and kept training. The new code refused to continue.

1 The code that stopped it (dataset.py)

```
if not os.path.isfile(fp):
  raise FileNotFoundError(
    f"Embedding not found for patch ...")
```

2 What it printed (errors.err)

```
FileNotFoundError: Embedding not found
for patch .../case_058_match_1_h&e_patch13.png
```

3 Why: the patch is too small

case_058 ... patch13 is **24 × 23 px**. The embedding script skips anything under 32 × 32, so no .pt file was ever made:

```
if w < 32 or h < 32:
  return ..., "too_small" # no .pt saved
```

4 Decision: exclude and list

Until the data is fixed, these patches are left out and every one is printed as **EXCLUDED**.

Split 1 training job (sacct: FAILED, 00:01:21). Size rule: precompute_pooled_features.py, MIN_PATCH_SIZE = 32, lines 66–69.

Finding · How many patches had no embedding

1,487
patches with no embedding, now excluded and listed

130
slices lost at least one patch

1,487 / 1,487
are smaller than 32 × 32 (checked every one)

Case	Stain	Slice	Patches excluded
47	H&E	match_1	124
70	Sox10	match_3	12
18	H&E	match_1	11
50	Melan-A	match_4	11
50	Sox10	match_5	11

Examples from the run log (grep EXCLUDED outputs.out)

These patches were missing in every earlier run too. The old code dropped them without a message, so nobody knew.

Now each one is printed as **EXCLUDED** in the log and saved in excluded.txt.

Split 1 run. Counts cover slices loaded in this run (at most 5 slices per stain), so the full count could be higher.

Results · Split 1 run, and what's next

1 The run finished

12,324 test patches from 18 test cases, each saved with its own score, filename and path in patch_attention_by_filename.csv.

Figures open each patch by that path, not by sort order.

Test accuracy: **16 of 18**.

2 Where it is

Code, log, excluded list and run results are in the team repo:

```
Presentation2/
Lily_Presentation2_code/
```

3 Next

Share excluded.txt so the shared data can be fixed.

Part 2: outline high- and low-attention patches on their slices with find_patch.py.

Analysts Choice 1 - Part 2

1 Before

Attention scores per patch, shown as individual pictures

→ No sense of where in the tissue the model was looking.

2 After

The top patches outlined in place on the whole slice

→ Each outline is labeled with its rank and weight.

Every score is now paired with its patch **by filename**, so the patch we outline is actually the patch that earned the score.

Code in https://github.com/arvindkrishna87/Team2_TheConsultancy/tree/main/Presentation2/Bella_Presentation2_code

How It Works

1. Read scores

patch_attention_ by_filename.csv from the training run

2. Pick top 5

Highest attention on each slice

3. Locate

find_patch matches each patch to the slice image

4. Outline

Red box, rank and weight; one image per slice

Code in https://github.com/arvindkrishna87/Team2_TheConsultancy/tree/main/Presentation2/Bella_Presentation2_code

Building on find_patch.py

What it does

1. Finds SIFT keypoints on the slice: distinctive tissue features that survive changes in scale and rotation
2. Finds keypoints on the patch and matches them to the slice
3. Fits the patch's position, scale and rotation, keeping only matches that agree (inliers)
4. Returns center, scale, rotation, inliers and confidence

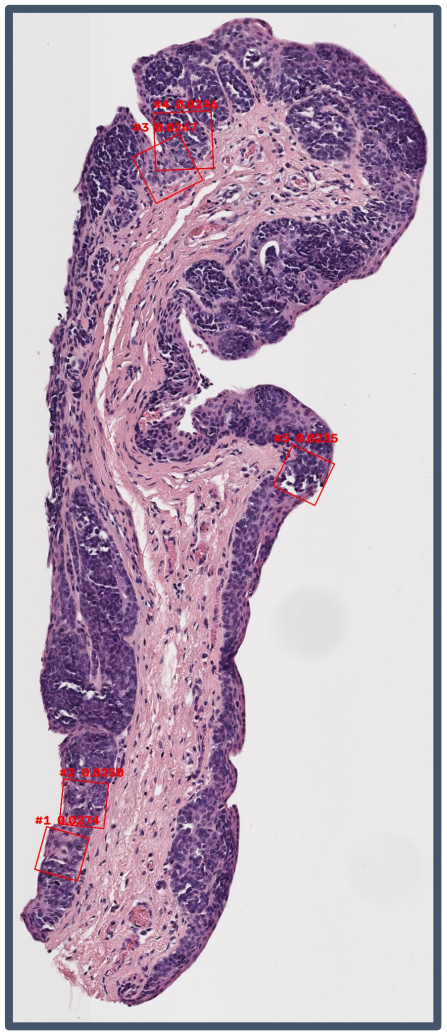
What I built on it

- Compute slice keypoints once and reuse them for 5 patches
- Turn center, scale and rotation into a box on the slice
- Draw all 5 boxes on one image, labeled with rank and weight

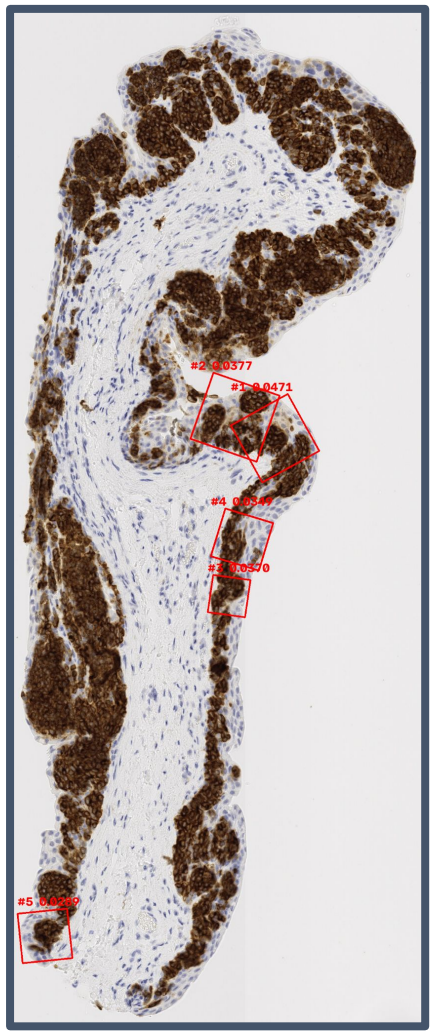
Fallbacks: If there are future unmatched patches, report as NOT FOUND instead of silently hiding them like before.

Code in https://github.com/arvindkrishna87/Team2_TheConsultancy/tree/main/Presentation2/Bella_Presentation2_code

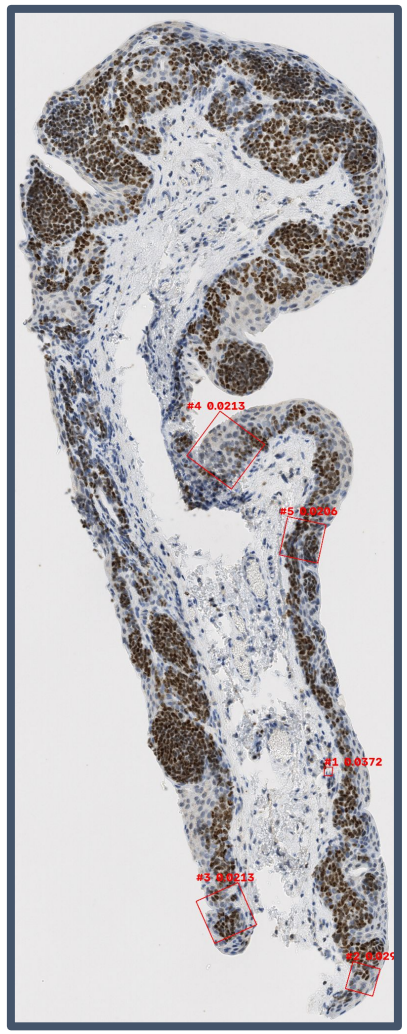
Case 1 Figures



Case 1 · H&E · case_1_match_1_h&e_top5.png



Case 1 · Melan-A · case_1_match_1_melan_top5.png



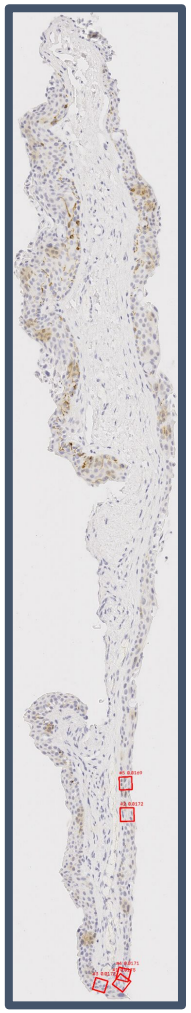
Case 1 · Sox10 · case_1_match_1_sox10_top5.png

All .tif files used and outputs in /projects/e32998/STAT390_Fall2026/Team2_TheConsultancy/Presentation2/Bella_Presentation2_code since too large to git push

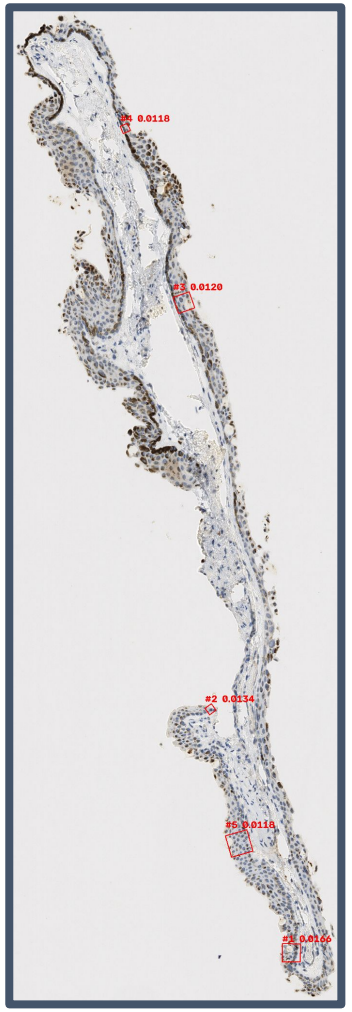
Case 4 Figures



Case 4 · H&E · case_4_match_1_h&e_top5.png



Case 4 · Melan-A · case_4_match_1_melan_top5.png



Case 4 · Sox10 · case_4_match_1_sox10_top5.png

All .tif files used and outputs in /projects/e32998/STAT390_Fall2026/Team2_TheConsultancy/Presentation2/Bella_Presentation2_code since too large to git push

What Got Saved

1. Slice Figures

patch_locations/<slice>_top5.png

Per slice: One image, top 5 outlined in red with rank and weight

2. Match Log

Terminal output for progress / checks

Per slice: Outlined 5/5 patches or NOT FOUND

Next Steps

1. Account for Nearby Patches

- Text currently overlaps / gets crowded

2. Incorporate Code into Pipeline

- Code should run after attention analysis, so every test slice gets a figure.

Code in https://github.com/arvindkrishna87/Team2_TheConsultancy/tree/main/Presentation2/Bella_Presentation2_code

All .tif files used and outputs in /projects/e32998/STAT390_Fall2026/Team2_TheConsultancy/Presentation2/Bella_Presentation2_code since too large to git push

Coders · Choice 1

Does UNI2 beat KimiaNet?

Frozen UNI2-h patch features vs the KimiaNet baseline, same hierarchical attention model, same 5 case splits

KimiaNet 0.837 → **UNI2 0.942** test accuracy, all 86 cases

One model, two frozen encoders

KimiaNet (baseline)

- DenseNet-121 CNN (2020), 7M parameters
- Trained on 240,000+ TCGA patches, H&E only
- 2×2 pooled map → 4,096 numbers/patch
- Any patch size accepted as is

UNI2-h

- Vision Transformer, 681M parameters
- Self-supervised: 200M+ H&E and IHC images
- CLS token → 1,536 numbers per patch
- Pretraining covers IHC: Melan-A, SOX10

HELD FIXED SO ONLY THE ENCODER DIFFERS

Same 5 splits, 86 cases

Same hyperparameters

Same patch → slice → stain → case attention

Test on lowest-val-loss checkpoint

Only the projector input changes: 4,096 → 512 vs 1,536 → 512

Trainable MIL parameters: 2,493,061 (KimiaNet) vs 1,182,341 (UNI2); both encoders frozen

UNI2 features: cost and preprocessing

81,627
patches embedded once, reused for every run

2,003
patches under 32 px skipped, same rule as KimiaNet

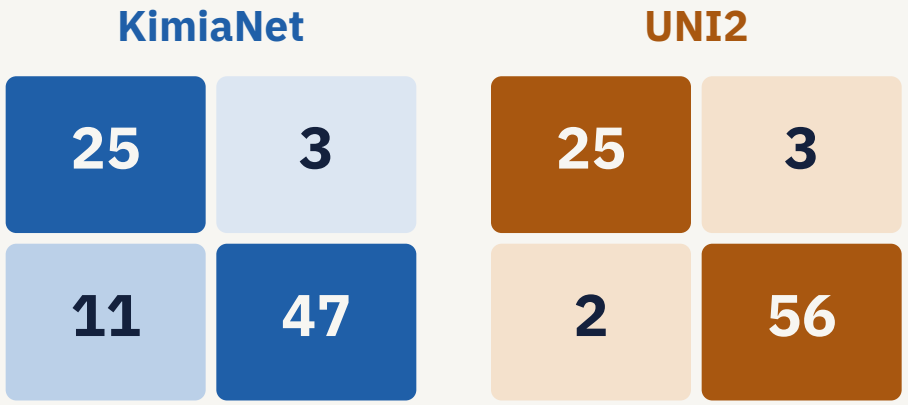
Patches are not all one size

Longest side (1,000 sampled)	px
Median	192
95th percentile	608
99th percentile	2,051

- No 224 × 224 crop: native size, snapped to multiples of 14 px (UNI2's token size)
- Longest side capped at 518 px (~7% of patches shrunk): attention cost grows with tokens², and a 2,048 px patch is ~21,000 tokens

Weights loaded with strict=True, so a wrong checkpoint is flagged instead of silently giving an untrained network

UNI2 catches the high-grade cases KimiaNet misses



All 86 test cases	KimiaNet	UNI2
Accuracy	0.837	0.942
Benign recall	0.893	0.893
High-grade recall	0.810	0.966
AUC	0.932	0.983

Matrices: rows = true (benign, high-grade), columns = predicted (benign, high-grade), pooled over the 5 test folds. Benign is identical (25 of 28); high-grade misses drop from 11 to 2.

UNI2 is ahead on all 5 splits

Split	n	KimiaNet	UNI2	Best ep. K/U
1	17	0.765	0.882	3 / 30
2	18	0.833	0.944	6 / 5
3	17	0.765	0.941	6 / 17
4	17	0.941	1.000	11 / 8
5	17	0.882	0.941	8 / 7
Mean	86	0.837 ± 0.076	0.942 ± 0.042	

PAIRED TEST, SAME 86 CASES

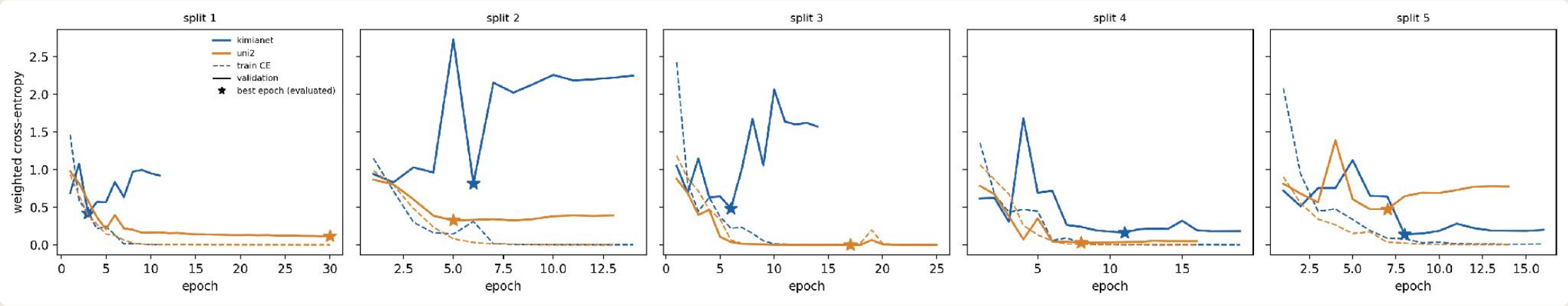
13 vs 4

cases right only with UNI2 vs only with KimiaNet (68 both, 1 neither)

Exact McNemar p = 0.049

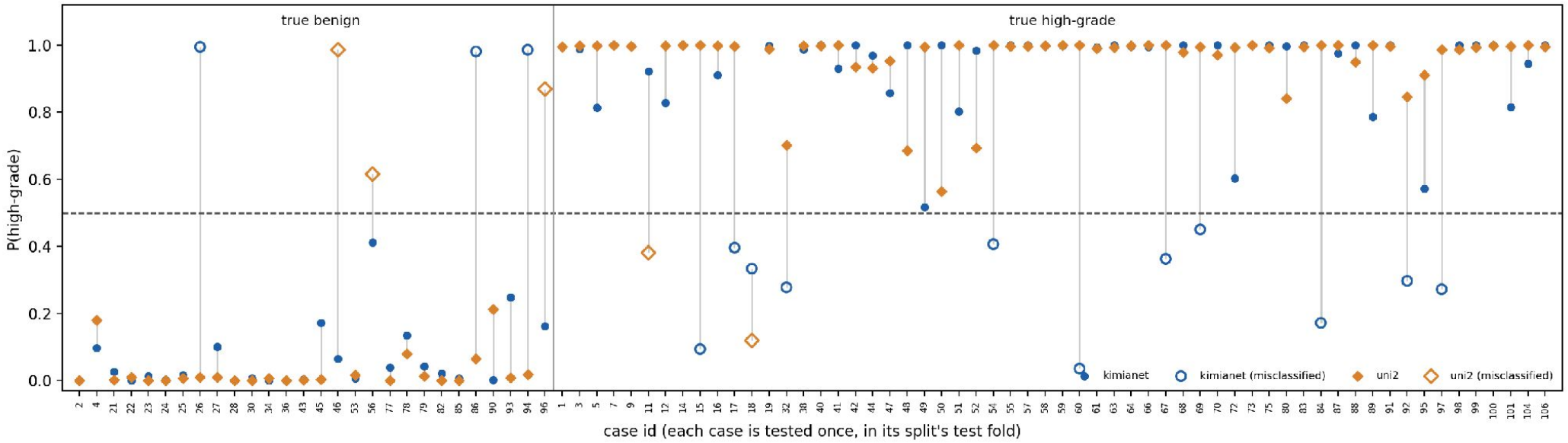
Evidence for UNI2, but borderline with 86 cases

Both memorize; UNI2's validation loss stays low



Top: per epoch (dashed = train CE, solid = validation, ★ = evaluated epoch). Bottom: per gradient update, 1 case each; spikes are single cases under class-weighted summed loss.

Every test case, both encoders



Rescued by UNI2

High-grade 15, 60, 84: P(HG) 0.04–0.17 → above 0.999

Confident errors

Benign 46 (0.99) and 96 (0.87) for UNI2; benign 26, 86, 94 (≥ 0.98) for KimiaNet

Missed by both

High-grade 18 (0.33 and 0.12). UNI2 is often 0.999+, so not calibrated

Analysis of these Results

CAVEATS

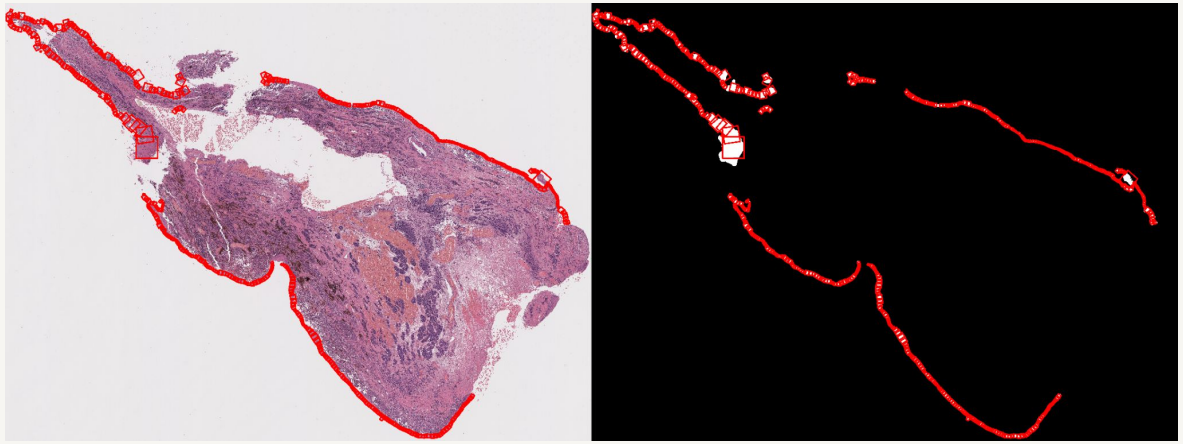
- Hyperparameters were tuned for KimiaNet, so if anything UNI2 is understated
- One training seed per split; p = 0.049 on 86 cases
- UNI2 split 1 peaked at epoch 30, the cap
- Patches over 518 px shrunk for UNI2 only
- Baseline is lower than the earlier 0.895: class weights now act (reduction="sum") and the best epoch is tested

POSSIBLE NEXT STEPS

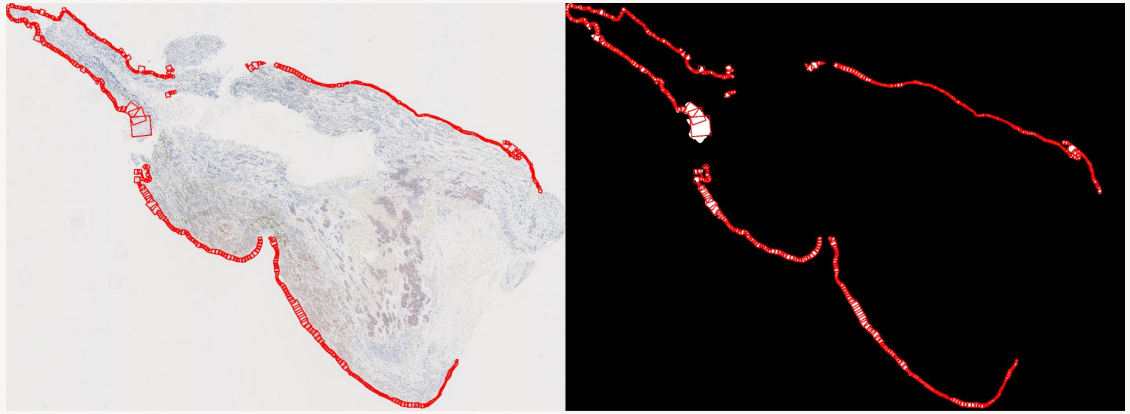
- Retune for UNI2, ideally with nested CV
- Several seeds per split; allow more epochs
- Calibrate probabilities
- Add the new cases (93) once embedded

Data Engineering

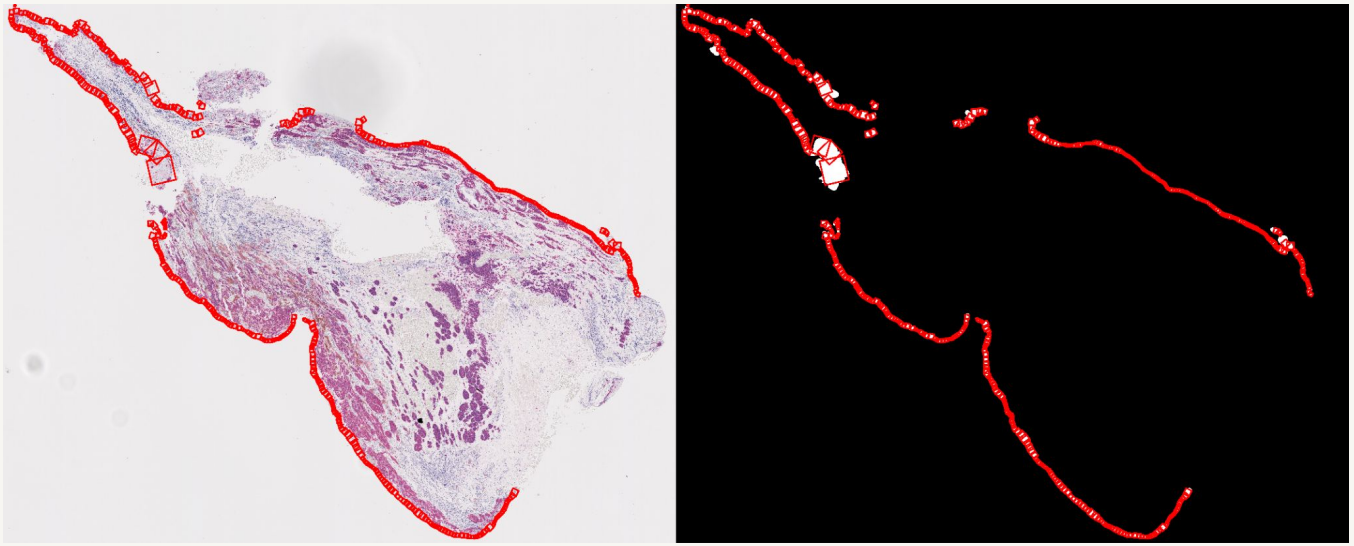
STEP 1 · Patching Case 62



h&e



Sox10



Melan

STEP 2 · STATUS

Vector embedding of new patches

5,372 patches → 5,372 patches

- Cases 8, 20, 33, 35, 62 and 102, in one folder, embeddings_cases_8_20_33_35_62_102
- Patch numbers checked before and after

Changes to precompute_embedding.sbatch

The embedding script pointed at a UNI2 checkpoint and placeholder paths → Now points to KimiaNet, the correct model

precompute_embedding.sbatch, line 35

STEP 3 · ADD VARIETY

Every training patch gets two flipped versions

Each epoch, each patch draws one at random

Patch embedding (81,627 patches)

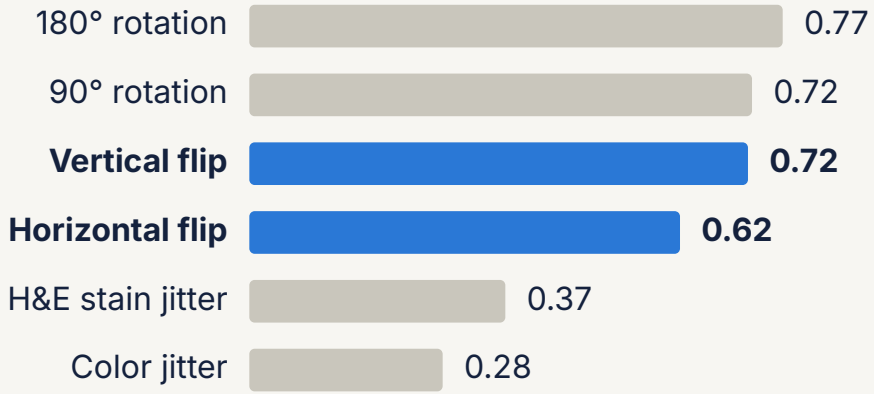
Original	Horizontal flip	Vertical flip
1 in 3	1 in 3	1 in 3

Flipped files sit in mirror folders with the same file names

- embeddings_aug/hflip/
- embeddings_aug/vflip/

Training only; validation and test always use originals.

Transformations decided out of a variety of options, measured by relative shift to the original patch

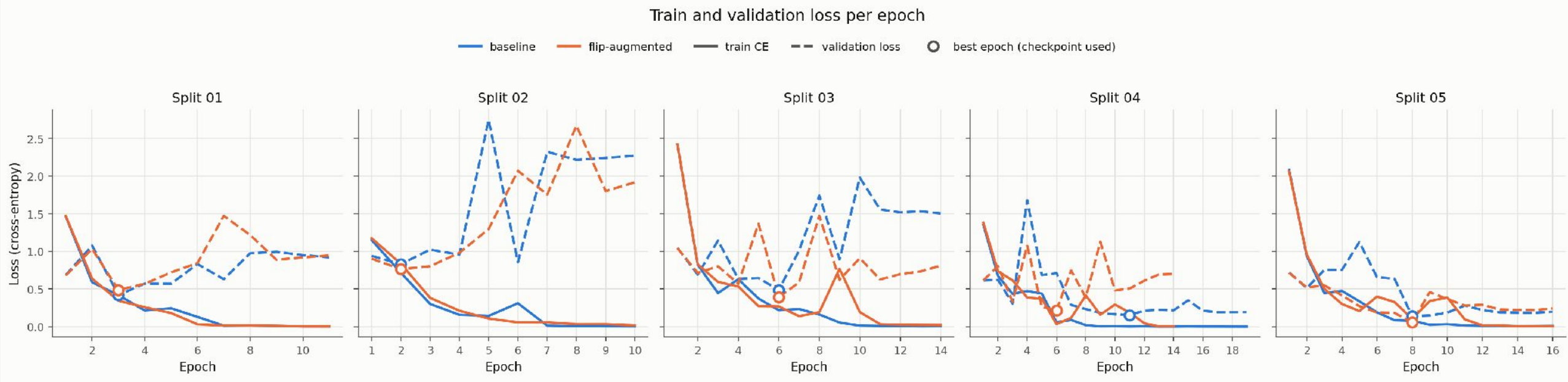


1.0 = as far as a different patch from the same slice. Blue = used. 500 random training patches selected, 491 tested (9 below the 32 pixel minimum)

Cost: 2 × 81,627 extra embeddings, about 3 h each on CPU, run in parallel. Training time per split unchanged (10–14 min).

STEP 4 · MEMORIZATION

Flips did not slow memorization



Final training loss (baseline vs flip)

0.004 vs 0.011

Epoch training loss drops below 0.05

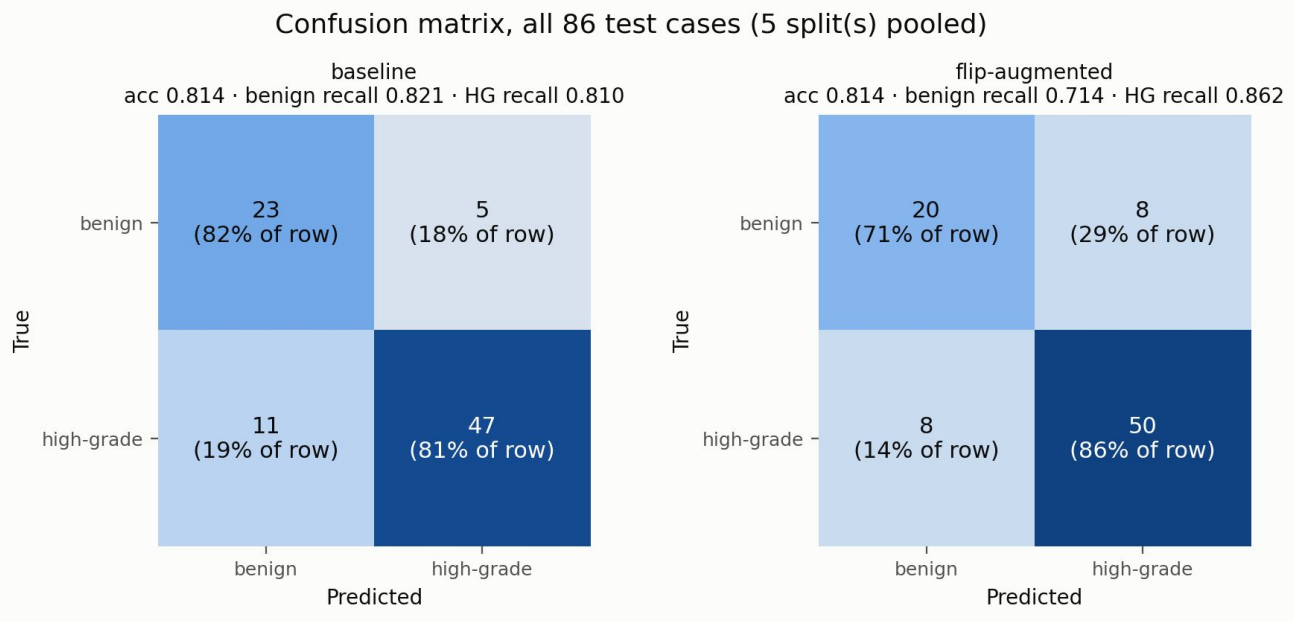
7–10 vs 6–12

Validation loss rise after best epoch

0.87 vs 1.06

STEP 4 · PERFORMANCE, 86 TEST CASES

Same accuracy, different mistakes

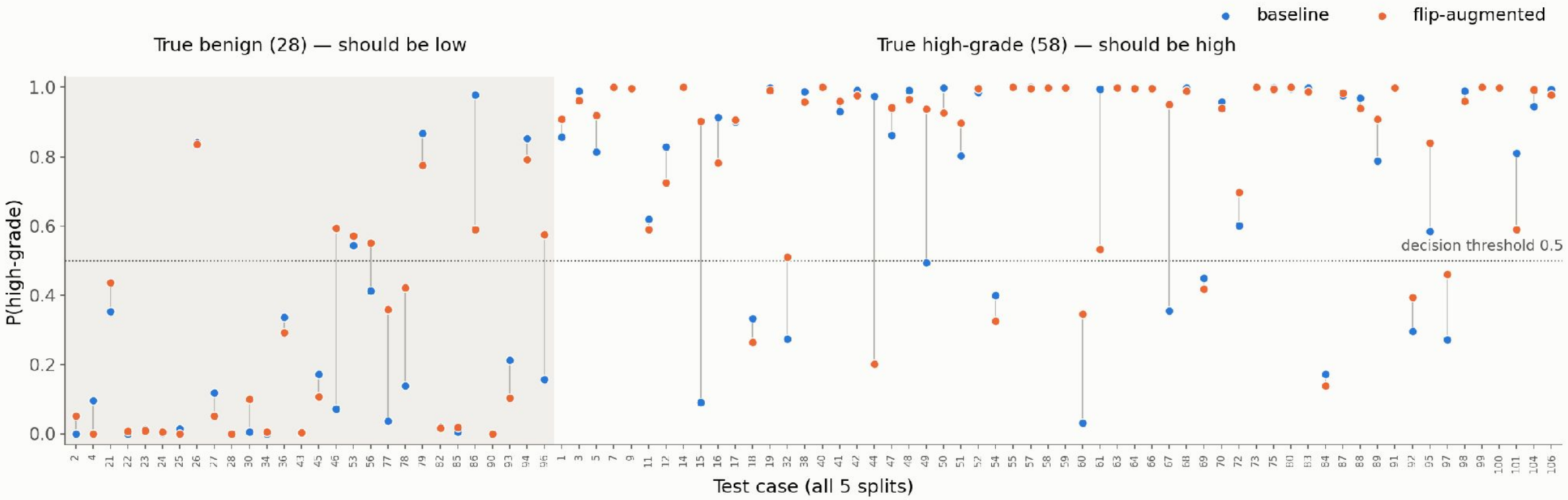


Metric	Baseline	Flip
Accuracy	0.814	0.814
Benign recall	0.821 (23/28)	0.714 (20/28)
High-grade recall	0.810 (47/58)	0.862 (50/58)
Only ½ models correct	4 cases	4 cases

Paired McNemar test p = 1.0: no real difference.

STEP 4 · EVERY TEST CASE

Individual cases swing a lot, with no net gain



Grey lines join the two models' predictions for the same case. Cases **15, 44, 46, 61, 67, 86 and 96** move by 0.4–0.8.

NEXT STEPS

Change the whole case, not single patches

Why flips likely failed

The model only has to tell about 52 training cases apart.

A flip changes single patches, but each case is a bag of hundreds of patches, so the case as a whole still looks the same.

Evidence: training loss reached about 0 by epoch 6–12, with or without flips.

Suggestions of next steps:

1 **Bag-level augmentation**

Each epoch, keep a random 50–70% of each slice's patches. No new embeddings needed.

2 **More training cases**

Add the new high-grade cases and move to the 93-case splits.

3 **Noise or mixing at the embedding level**

Add small random noise to embeddings, or blend patches from two cases of the same class.

Test each against the same baseline and 5 splits, with 3 seeds, since one test case is worth about 6% accuracy per split.

Patch Size Handling

The problem · UNI2 is set up for 224 px; our patches vary

59% of our patches are smaller than 224 px, and patch size differs by grade

1 · Input image size

UNI2's model card sets the model up for 224 × 224 input; its training details are not published. KimiaNet was trained on 1000 × 1000 px patches.

2 · Variable-sized images

KimiaNet takes each patch at its native size. UNI2 needs a choice: resize every patch, or keep native size.

Our 83,630 patches: median side 195 px, largest 2,053 px (2,048 cap plus rotation and crop)

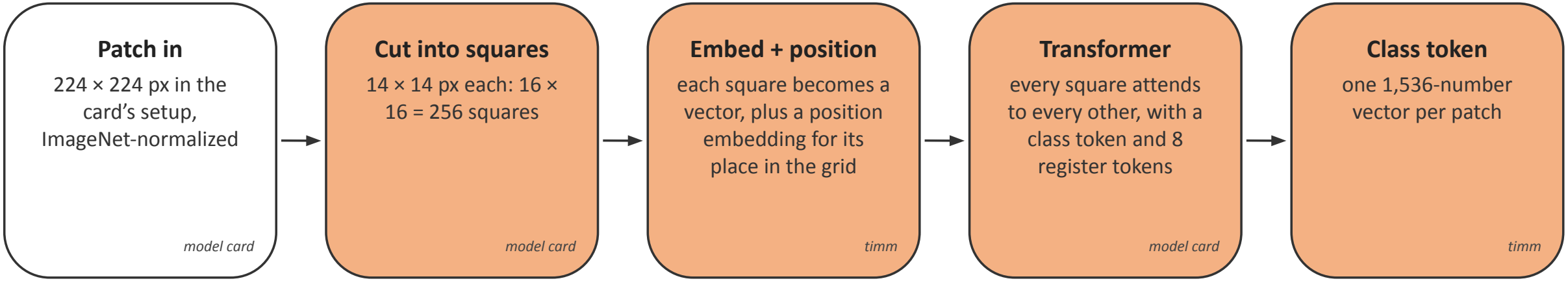


Large patches skew high-grade: 4.4% of high-grade patches are 1,024 px or larger, vs 0.2% of benign.
Resizing every patch to 224 px could erase a size difference that tracks grade.

Source: UNI2-h model card; KimiaNet paper; Quest reports uni2_input_check.md, folder_check.md

UNI2 · How the transformer reads a patch

The patch is cut into a fixed grid of squares, and each square gets a position embedding for its place in the grid



Model size

ViT-H with about 681M parameters

Pretraining data

200M+ tiles from H&E and IHC slides

Why size matters: at 224 px the model sees a 16 × 16 grid, and its position embeddings are set up for that grid

Source: UNI2-h model card (MahmoodLab/UNI2-h); timm v1.0.30 source code (read, not run)

UNI2 · What happens when a patch is not 224 px

Other sizes work only if each side divides by 14, so we would have to pad patches ourselves

Fit the grid

Height and width must each divide by 14, or timm raises an error. Padding (dynamic_img_pad) is off by default.

timm

→

Stretch the positions

With dynamic_img_size on, as in the card, the 16 × 16 position embeddings are resampled (bicubic) to the new grid

timm

→

Still one vector

The class token is the default output, so any valid size gives one 1,536-number vector

timm

Patch side	Grid	Squares
195 px (our median)	14 × 14 (padded to 196)	196
224 px (card setup)	16 × 16	256
448 px	32 × 32	1,024
2,048 px cap (pad to 2,058)	147 × 147	**21,609**

Source: timm v1.0.30 source code (read, not run); UNI2-h model card

Comparison · KimiaNet vs two ways to feed UNI2

Resizing matches the model card's setup; native size keeps true scale

KimiaNet today

Baseline

Native size, pooled to 4,096 numbers

+ True scale kept for every patch

− Cost grows with patch size

− Skips patches under 32 × 32 px (2,003 patches)

UNI2 A

Resize to 224

Every patch becomes 224 × 224

+ Matches the card's preprocessing

+ Fixed cost: always 256 squares

− Scale varies: patches under 32 px enlarged 7× or more, largest shrunk about 9×

UNI2 B

Native, padded

Padded by us to a multiple of 14

+ True scale kept

− Up to 21,609 squares, about 84× more than A

− Grids the card doesn't use; positions resampled

− Patches 14 px or smaller become one square

A matches the model card's setup but distorts scale. B keeps true scale but costs more and is untested at these grid sizes.

Source: UNI2-h model card; timm v1.0.30; KimiaNet paper; Code_walkthrough.pdf

Comparison · What the evidence suggests

Scale matters for these models, but no source says which option is better for UNI2 on our patches

Mind the Gap

Scale matters

"Magnification is a primary driver of performance variation across models."

UNI, trained mostly at 0.5 mpp, drops as test magnification moves away from it.

UNI-v2 does best at high magnification (0.25 to 0.5 mpp), but its training details are not disclosed.

Other papers

Fixed scale

PRISM2 tiles 224 × 224 px at 20× (0.5 mpp).

The course's H&E/IHC paper tiles 224 × 224 px at ×10 (about 1.0 micron per pixel).

Both fix the physical scale, not just the pixel size.

Our KimiaNet runs

Hint only

Resized to 224 vs native size, same split, 19 test cases:

Accuracy 17/19 for both; test loss 0.395 resized vs 0.494 native.

Not identical inputs: the native run skipped 2,003 patches under 32 px.

Resizing did not hurt KimiaNet on one small split, but that does not tell us how UNI2 will behave.

Source: Mind the Gap (arXiv 2601.02198); PRISM2 (arXiv 2506.13063); Synergistic H&E/IHC paper

Recommendation · Test both options, then choose

A is the model card's documented setup; B is the experiment to test against it

1 · Embed both ways
frozen UNI2: A (resize to 224) and B (pad to a multiple of 14)

→

2 · Train
same 5 splits, KimiaNet as the baseline

→

3 · Compare
accuracy, benign and high-grade recall, AUC per split

→

4 · Choose
the option that wins across splits, not on one

Limitations

Open questions

- UNI2's training scale is not published, and our patches keep the 40× scan resolution, so we can't say whether A or B is closer to UNI2's training
- No UNI2 runs of our own yet
- In-house comparison is one split of 19 cases
- B processes up to about 84× more squares than A; speed untested

Source: UNI2-h model card; timm v1.0.30; 40× scan resolution assumes our slides are the WHO validation study's 40× scans (same 105 cases) exported at full resolution per Code_walkthrough.pdf